
File Handling in Python

File handling is crucial for any application that needs to store or retrieve data permanently. Python provides a rich set of tools to interact with files on your computer's file system.

Types of Files

There are two primary types of files you'll work with:

1. **Text Files (.txt, .csv, .py, .html):** These files store data as human-readable characters. Each line of text is terminated with a special character called an End of Line (EOL) character (like `\n`). They are easy to read and edit with a simple text editor.
2. **Binary Files (.jpg, .mp4, .dat, .exe):** These files store data as a sequence of bytes (0s and 1s), which is how a computer stores all data. They are not human-readable and require a specific program to interpret their content correctly. Examples include images, videos, and executable files.

Creating and Reading Text Data

To work with a file, you first need to open it using the built-in `open()` function. The best practice is to use the `with` statement, which automatically handles closing the file for you, even if errors occur.

Syntax: `open(file_path, mode)`

- **file_path:** The name and location of the file.

- **mode:** A string indicating how the file will be used. Common modes include:
 - 'r': **Read** (default). Fails if the file doesn't exist.
 - 'w': **Write**. Creates a new file or overwrites an existing one.
 - 'a': **Append**. Adds content to the end of an existing file. Creates the file if it doesn't exist.
 - 'x': **Create**. Exclusively creates a new file; fails if it already exists.
 - '+': Can be added to a mode (e.g., 'r+', 'w+') to allow both reading and writing.

Example: Creating and Writing to a File

Python

```
# The 'w' mode creates 'greeting.txt' and lets us write to it.
with open('greeting.txt', 'w') as f:
    f.write("Hello, Python!\n")
    f.write("This is a new line.")

print("File 'greeting.txt' created and written to successfully.")
```

Output:

File 'greeting.txt' created and written to successfully.

(This creates a file named greeting.txt with two lines of text.)

Example: Reading from a File

```
# The 'r' mode opens the file for reading.
with open('greeting.txt', 'r') as f:
    content = f.read()
    print("Content of the file:")
    print(content)
```

Output:Content of the file:

Hello, Python!

This is a new line.

File Methods to Read and Write Data

Python provides several methods to read and write data efficiently.

Writing Methods:

- `write(string)`: Writes a single string to the file.
- `writelines(list_of_strings)`: Writes each string from a list or iterable to the file. Note: It does not add newline characters automatically.

Reading Methods:

- `read(size)`: Reads size bytes from the file. If size is omitted, it reads the entire file.
- `readline()`: Reads a single line from the file, including the newline character `\n`.
- `readlines()`: Reads all lines from the file and returns them as a list of strings.

Example: Using `readlines()` and `writelines()`

Python

```
lines_to_write = ["First line\n", "Second line\n", "Third line\n"]
```

```
# Writing a list of lines to a file
```

```
with open('multiple_lines.txt', 'w') as f:  
    f.writelines(lines_to_write)
```

```
# Reading all lines into a list
```

```
with open('multiple_lines.txt', 'r') as f:  
    lines_read = f.readlines()  
    print("Lines read from file:", lines_read)
```

Output:

```
Lines read from file: ['First line\n', 'Second line\n', 'Third line\n']
```

Reading and Writing Binary Files

To work with binary files, you simply add 'b' to the mode string (e.g., 'rb' for reading binary, 'wb' for writing binary).

Example: Writing and Reading a sequence of bytes

Python

```
# Some binary data (a sequence of bytes)
```

```
data_to_write = b'\x01\x02\x03\x04\x05'
```

```
# Writing binary data
```

```
with open('data.bin', 'wb') as f:
```

```
    f.write(data_to_write)
```

```
    print("Binary file 'data.bin' created.")
```

```
# Reading binary data
```

```
with open('data.bin', 'rb') as f:
```

```
    data_read = f.read()
```

```
    print("Binary data read:", data_read)
```

```
    print("As a list of integers:", list(data_read))
```

Output:

Binary file 'data.bin' created.

Binary data read: b'\x01\x02\x03\x04\x05'

As a list of integers: [1, 2, 3, 4, 5]

A **bytes** object is an **immutable sequence of single bytes**. The key concept here is that Python treats each individual byte in this sequence as an **integer** with a value between 0 and 255.

- `\x01` is the byte representing the integer **1**.
- `\x02` is the byte representing the integer **2**.
- And so on.

So, the process looks like this:

b'\x01\x02\x03\x04\x05' → 1, 2, 3, 4, 5 → [1, 2, 3, 4, 5]

The pickle Module

The pickle module is fantastic for **serialization**, which is the process of converting a Python object (like a list, dictionary, or custom object) into a byte stream that can be stored in a file. This allows you to save your program's state and load it back later.

- `pickle.dump(obj, file)`:
Serializes the object `obj` and writes it to the file object `file` (opened in binary write mode 'wb').
- `pickle.load(file)`:
Reads a serialized object from the file object `file` (opened in binary read mode 'rb') and reconstructs it.

Example: Saving and Loading a Dictionary

```
import pickle

my_data = {
    'name': 'Alex',
    'id': 123,
    'courses': ['Math', 'Physics', 'Computer Science']
}

# Save the dictionary to a file
with open('user_data.pkl', 'wb') as f:
    pickle.dump(my_data, f)
    print("Dictionary pickled and saved to 'user_data.pkl'")

# Load the dictionary back from the file
with open('user_data.pkl', 'rb') as f:
    loaded_data = pickle.load(f)
    print("Data loaded from pickle file:")
    print(loaded_data)

print("Is the loaded object the same?", loaded_data == my_data)
```

Output:

```
Dictionary pickled and saved to 'user_data.pkl'
Data loaded from pickle file:
```

```
{'name': 'Alex', 'id': 123, 'courses': ['Math', 'Physics', 'Computer Science']}
```

Is the loaded object the same? True

Reading and Writing CSV Files

CSV (Comma-Separated Values) files are a common format for storing tabular data (like spreadsheets). Python's csv module makes it easy to work with them.

Example: Writing and Reading a CSV File

```
import csv

# Data to be written (list of lists)
student_data = [
    ['Name', 'Age', 'Grade'],
    ['Alice', 21, 'A'],
    ['Bob', 22, 'B'],
    ['Charlie', 20, 'A']
]

# Writing to a CSV file
# newline="" prevents extra blank rows from being added
with open('students.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(student_data)
    print("Data written to students.csv")

# Reading from a CSV file
with open('students.csv', 'r') as f:
```

```
reader = csv.reader(f)
print("\nReading data from students.csv:")
for row in reader:
    print(row)
```

Output:

Data written to students.csv

Reading data from students.csv:

['Name', 'Age', 'Grade']

['Alice', '21', 'A']

['Bob', '22', 'B']

['Charlie', '20', 'A']

Python os and os.path Modules

These modules provide a way to interact with the operating system, allowing you to perform file system operations like creating directories, renaming files, and checking if paths exist.

- **os module:** Interacts directly with the operating system (e.g., managing files and directories).
- **os.path module:** A submodule of os for manipulating pathnames.

Example: Common Operations

Python


```
import os

# Get current working directory
print(f"Current Directory: {os.getcwd()}")

# Check if a file exists
file_path = 'greeting.txt'
if os.path.exists(file_path):
    print(f"'{file_path}' exists.")
    # Rename the file
    os.rename(file_path, 'salutation.txt')
    print(f"Renamed '{file_path}' to 'salutation.txt'")

# List files and directories in the current directory
print(f"\nFiles in current directory: {os.listdir('.')}")

# Remove a file
if os.path.exists('salutation.txt'):
    os.remove('salutation.txt')
    print("\n'salutation.txt' has been removed.")
```

Output (will vary slightly based on your environment):

```
Current Directory: /path/to/your/project
'greeting.txt' exists.
Renamed 'greeting.txt' to 'salutation.txt'
```

```
Files in current directory: ['multiple_lines.txt', 'students.csv', 'data.bin', 'user_data.pkl',
'salutation.txt']
```

'salutation.txt' has been removed.

Object-Oriented Programming (OOP) in Python

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data (attributes) and code (methods). It helps organize complex software into reusable and logical blocks.

Classes and Objects

- **Class:** A blueprint or template for creating objects. It defines a set of attributes and methods that the created objects will have. For example, a Car class would define that all cars have attributes like color and model, and methods like start_engine().
- **Object (Instance):** An instance of a class. It's the actual thing created from the blueprint. For example, my_blue_tesla would be an object of the Car class.

Creating Classes and Objects in Python

You create a class using the class keyword. You create an object by calling the class name as if it were a function.

Python

```
# Creating a simple class
class Dog:
    # A simple method
```

```
def bark(self):  
    return "Woof!"
```

```
# Creating an object (instance) of the Dog class  
my_dog = Dog()
```

```
# Calling a method on the object  
print(my_dog.bark())
```

Output:

```
Woof!
```

The Constructor Method (__init__)

The `__init__` method is a special method called a **constructor**. It is automatically called when a new object is created. Its primary job is to initialize the object's attributes. The `self` parameter refers to the instance of the object itself, allowing you to attach data to it.

Python

```
class Car:  
    # Constructor method  
    def __init__(self, color, model):  
        # These are data attributes (instance variables)  
        self.color = color  
        self.model = model  
        print(f"A {self.color} {self.model} has been created!")
```

```
# Creating objects (instances) and passing arguments to the constructor
car1 = Car("Blue", "Sedan")
car2 = Car("Red", "SUV")
```

Output:

```
A Blue Sedan has been created!
A Red SUV has been created!
```

Classes with Multiple Objects

You can create as many objects from a single class as you need. Each object is independent and has its own set of data attributes.

Python

```
# Using the Car class from the previous example
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model

car1 = Car("Blue", "Sedan")
car2 = Car("Red", "SUV")

# Each object has its own data
print(f"Car 1 is a {car1.color} {car1.model}")
print(f"Car 2 is a {car2.color} {car2.model}")
```

Output:

Car 1 is a Blue Sedan

Car 2 is a Red SUV

Class Attributes vs. Data Attributes

- **Data Attributes (Instance Attributes):** These are owned by a specific instance of a class. They are defined inside the `__init__` method using `self.attribute_name`. Each object has its own copy of these attributes. (e.g., `car1.color` is different from `car2.color`).
- **Class Attributes:** These are shared by all instances of a class. They are defined directly inside the class, outside of any method.

Python

```
class Pet:
```

```
    # Class attribute (shared by all pets)
```

```
    species = "Canis familiaris"
```

```
    def __init__(self, name):
```

```
        # Data attribute (unique to each pet)
```

```
        self.name = name
```

```
pet1 = Pet("Buddy")
```

```
pet2 = Pet("Lucy")
```

```
# Accessing data attributes
```

```
print(f'{pet1.name}'s species is {pet1.species}) # Accessing class attribute via
```

```
instance
print(f"{pet2.name}'s species is {pet2.species}")

# Accessing class attribute directly via the class
print(f"All pets in this class belong to the species: {Pet.species}")
```

Output:

```
Buddy's species is Canis familiaris
Lucy's species is Canis familiaris
All pets in this class belong to the species: Canis familiaris
```

Encapsulation

Encapsulation is the bundling of data (attributes) and the methods that operate on that data into a single unit (a class). It's also about restricting direct access to an object's components, which is a key principle for preventing accidental data modification.

In Python, this is achieved by convention using underscores:

- `_variable`: A "protected" attribute. It's a hint to other programmers that it's intended for internal use and shouldn't be modified directly from outside the class.
- `__variable`: A "private" attribute. Python performs **name mangling**, making it harder to access from outside (e.g., `__balance` becomes `_ClassName__balance`).

Python

```
class BankAccount:
```

```

def __init__(self, initial_balance):
    # Private attribute
    self.__balance = initial_balance

def deposit(self, amount):
    if amount > 0:
        self.__balance += amount
        print(f"Deposited ${amount}. New balance: ${self.__balance}")

def get_balance(self):
    # A "getter" method to safely access the private attribute
    return self.__balance

# Create an account
my_account = BankAccount(1000)

# We can't access __balance directly
# print(my_account.__balance) # This would raise an AttributeError

# We use the public methods to interact with the data
print(f"Initial Balance: ${my_account.get_balance()}")
my_account.deposit(500)

```

Output:

```

Initial Balance: $1000
Deposited $500. New balance: $1500

```

Inheritance 👥

Inheritance allows a new class (the **child** or **subclass**) to inherit attributes and methods from an existing class (the **parent** or **superclass**). This promotes code

reuse.

- The child class can use all the functionality of the parent class.
- It can add its own new functionality.
- It can **override** parent methods to provide its own specific implementation.
- The `super()` function is used to call a method from the parent class.

Python

```
# Parent class
```

```
class Animal:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def speak(self):
```

```
        raise NotImplementedError("Subclass must implement this method")
```

```
# Child class inheriting from Animal
```

```
class Cat(Animal):
```

```
    def speak(self): # Overriding the parent's speak method
```

```
        return "Meow"
```

```
# Another child class
```

```
class Dog(Animal):
```

```
    def __init__(self, name, breed):
```

```
        super().__init__(name) # Call the parent's constructor
```

```
        self.breed = breed
```

```
    def speak(self): # Overriding
```

```
        return "Woof"
```

```
my_cat = Cat("Whiskers")
```

```
my_dog = Dog("Fido", "Golden Retriever")
```

```
print(f"{my_cat.name} says: {my_cat.speak()}")
```

```
print(f"{my_dog.name} the {my_dog.breed} says: {my_dog.speak()}")
```


Output:

Whiskers says: Meow

Fido the Golden Retriever says: Woof

Polymorphism

Polymorphism (from Greek, meaning "many forms") is the ability of different objects to respond to the same method call in their own unique way. In the example above, both Cat and Dog objects have a `speak()` method, but each one implements it differently.

This allows you to write generic code that can work with different types of objects without needing to know their specific class.

Example: Demonstrating Polymorphism

Python

```
# Using the Animal, Cat, and Dog classes from the previous example
```

```
# Create a list of different animal objects
```

```
animals = [Cat("Whiskers"), Dog("Fido", "Golden Retriever"), Cat("Shadow")]
```

```
# Loop through the list and call the same method on each object
```

```
for animal in animals:
```

```
    # Python doesn't care if it's a Cat or a Dog,
```

```
    # it just calls the .speak() method that exists on the object.
```

```
    print(f"{animal.name} goes {animal.speak()}")
```

Output:

```
Whiskers goes Meow  
Fido goes Woof  
Shadow goes Meow
```

Sample Experiments

Here are the solutions to the programming problems.

1. Sort Words in a File

Problem: Write a program to sort words in a file and put them in another file. The output file should have only lower-case words.

Setup: Create a file named source.txt with the following content:

```
Python is an amazing Programming language  
It is versatile AND powerful
```

Solution:

```
Python
```

1. Sort words in a file

try:

```
with open('source.txt', 'r') as source_file:
```

```
    # Read the entire file and split into words
```

```
    all_text = source_file.read()
```

```
    words = all_text.split()
```

```
    # Convert all words to lower case
```

```
    lower_case_words = [word.lower() for word in words]
```

```
    # Sort the words alphabetically
```

```
    lower_case_words.sort()
```

```
with open('sorted_words.txt', 'w') as output_file:
```

```
    # Write each word on a new line
```

```
    for word in lower_case_words:
```

```
        output_file.write(word + '\n')
```

```
print("Words from 'source.txt' have been sorted and saved to 'sorted_words.txt'")
```

except FileNotFoundError:

```
    print("Error: 'source.txt' not found. Please create it first.")
```

Result: A new file sorted_words.txt will be created with the following content:

amazing

an

and

is

is

it

language

powerful

programming

python
versatile

2. Reverse Each Line of a File

Problem: Python program to print each line of a file in reverse order.

Setup: Create a file named lines.txt with the following content:

Hello World
Reverse this line
12345

Solution:

Python

2. Reverse each line of a file

```
try:
    with open('lines.txt', 'r') as f:
        print("Lines from 'lines.txt' in reverse:")
        for line in f:
            # strip() removes leading/trailing whitespace (like the newline char)
            #[::-1] is a slice that reverses the string
            reversed_line = line.strip()[::-1]
            print(reversed_line)

except FileNotFoundError:
    print("Error: 'lines.txt' not found. Please create it first.")
```

Output:

```
Lines from 'lines.txt' in reverse:  
dlroW olleH  
enil siht esreveR  
54321
```

3. Compute Characters, Words, and Lines in a File

Problem: Python program to compute the number of characters, words, and lines in a file.

Setup: Create a file named stats.txt with the following content:

```
This is the first line.  
And this is the second.
```

Solution:

Python

```
# 3. Compute file statistics
```

```
num_chars = 0  
num_words = 0
```

```

num_lines = 0

try:
    with open('stats.txt', 'r') as f:
        for line in f:
            num_lines += 1
            words_in_line = line.split()
            num_words += len(words_in_line)
            num_chars += len(line)

    print("File Statistics for 'stats.txt':")
    print(f"Number of Lines: {num_lines}")
    print(f"Number of Words: {num_words}")
    print(f"Number of Characters (including newlines): {num_chars}")

except FileNotFoundError:
    print("Error: 'stats.txt' not found. Please create it first.")

```

Output:

```

File Statistics for 'stats.txt':
Number of Lines: 2
Number of Words: 9
Number of Characters (including newlines): 48

```

4. Array (List) Operations

Problem: Write a program to create, display, append, insert, and reverse the order of the items in an array. (We'll use a Python list).

Solution:

Python

4. Array (List) Operations

Create a list

```
my_list = [10, 20, 40, 50]  
print(f"1. Initial List: {my_list}")
```

Append an item to the end

```
my_list.append(60)  
print(f"2. After Appending 60: {my_list}")
```

Insert an item at a specific index (index 2)

```
my_list.insert(2, 30)  
print(f"3. After Inserting 30 at index 2: {my_list}")
```

Reverse the order of the items

```
my_list.reverse()  
print(f"4. After Reversing the list: {my_list}")
```

Result (Output):

1. Initial List: [10, 20, 40, 50]

2. After Appending 60: [10, 20, 40, 50, 60]

3. After Inserting 30 at index 2: [10, 20, 30, 40, 50, 60]

4. After Reversing the list: [60, 50, 40, 30, 20, 10]

5. Matrix Operations

Problem: Write a program to add, transpose, and multiply two matrices.

Solution:

Python

5. Matrix Operations

```
def add_matrices(A, B):
    """Adds two matrices A and B."""
    # Ensure matrices have the same dimensions
    if len(A) != len(B) or len(A[0]) != len(B[0]):
        return "Matrices must have the same dimensions to add."

    result = [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]
    return result

def transpose_matrix(A):
    """Transposes a matrix A."""
    result = [[A[j][i] for j in range(len(A))] for i in range(len(A[0]))]
    return result

def multiply_matrices(A, B):
    """Multiplies two matrices A and B."""
    # Ensure A's columns match B's rows
    if len(A[0]) != len(B):
        return "Matrix A's columns must match Matrix B's rows to multiply."

    result = [[sum(a * b for a, b in zip(A_row, B_col)) for B_col in zip(*B)] for A_row in A]
    return result

# Sample Matrices
X = [[1, 2],
      [3, 4]]

Y = [[5, 6],
```



```

    [7, 8]]

Z = [[1, 2, 3],
     [4, 5, 6]]

# --- Addition ---
print("Matrix Addition (X + Y):")
add_result = add_matrices(X, Y)
for row in add_result:
    print(row)

# --- Transposition ---
print("\nMatrix Transpose (Z):")
transpose_result = transpose_matrix(Z)
for row in transpose_result:
    print(row)

# --- Multiplication ---
print("\nMatrix Multiplication (X * Y):")
mult_result = multiply_matrices(X, Y)
for row in mult_result:
    print(row)

```

Result (Output):

Matrix Addition (X + Y):
 [6, 8]
 [10, 12]

Matrix Transpose (Z):
 [1, 4]
 [2, 5]
 [3, 6]

Matrix Multiplication (X * Y):

[19, 22]

[43, 50]

6. OOP Shape Class Hierarchy

Problem: Write a Python program to create a class that represents a shape. Include methods to calculate its area and perimeter. Implement subclasses for different shapes like circle, triangle, and square.

Solution:

Python

```
# 6. OOP Shape Classes
```

```
import math
```

```
# Parent (Base) Class
```

```
class Shape:
```

```
    """A base class for all shapes."""
```

```
    def area(self):
```

```
        raise NotImplementedError("Subclasses must implement this method")
```

```
    def perimeter(self):
```

```
        raise NotImplementedError("Subclasses must implement this method")
```

```
# Subclass for Circle
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        if radius < 0:
```

```
            raise ValueError("Radius cannot be negative.")
```

```
        self.radius = radius
```

```
    def area(self):
```

```
# Area =  $\pi * r^2$   
return math.pi * (self.radius ** 2)
```

```
def perimeter(self):  
    # Perimeter (Circumference) =  $2 * \pi * r$   
    return 2 * math.pi * self.radius
```

```
# Subclass for Square
```

```
class Square(Shape):  
    def __init__(self, side_length):  
        self.side = side_length
```

```
    def area(self):  
        # Area = side * side  
        return self.side ** 2
```

```
    def perimeter(self):  
        # Perimeter =  $4 * \text{side}$   
        return 4 * self.side
```

```
# Subclass for Triangle (Right-Angled for simplicity)
```

```
class RightTriangle(Shape):  
    def __init__(self, base, height):  
        self.base = base  
        self.height = height  
        # Hypotenuse using Pythagorean theorem:  $a^2 + b^2 = c^2$   
        self.hypotenuse = math.sqrt(base**2 + height**2)
```

```
    def area(self):  
        # Area =  $0.5 * \text{base} * \text{height}$   
        return 0.5 * self.base * self.height
```

```
    def perimeter(self):  
        # Perimeter = base + height + hypotenuse  
        return self.base + self.height + self.hypotenuse
```

```
# --- Creating and using objects ---
```

```
my_circle = Circle(10)  
my_square = Square(6)
```

```

my_triangle = RightTriangle(3, 4)

print("--- Circle ---")
print(f"Radius: {my_circle.radius}")
print(f"Area: {my_circle.area():.2f}") # Format to 2 decimal places
print(f"Perimeter: {my_circle.perimeter():.2f}")

print("\n--- Square ---")
print(f"Side Length: {my_square.side}")
print(f"Area: {my_square.area()}")
print(f"Perimeter: {my_square.perimeter()}")

print("\n--- Right Triangle ---")
print(f"Base: {my_triangle.base}, Height: {my_triangle.height}")
print(f"Area: {my_triangle.area()}")
print(f"Perimeter: {my_triangle.perimeter()}")

# Demonstrating polymorphism
shapes = [Circle(5), Square(4), RightTriangle(6, 8)]
print("\n--- Polymorphism Demo ---")
for i, shape in enumerate(shapes):
    print(f"Shape {i+1} Area: {shape.area():.2f}, Perimeter: {shape.perimeter():.2f}")

```

Result (Output):

```

--- Circle ---
Radius: 10
Area: 314.16
Perimeter: 62.83

--- Square ---
Side Length: 6
Area: 36
Perimeter: 24

```

--- Right Triangle ---

Base: 3, Height: 4

Area: 6.0

Perimeter: 12.0

--- Polymorphism Demo ---

Shape 1 Area: 78.54, Perimeter: 31.42

Shape 2 Area: 16.00, Perimeter: 16.00

Shape 3 Area: 24.00, Perimeter: 24.00